

Arquitectura de Microservicios con Contenedores

- Ing. Federico Castañeda
- Ing. Carlos Maceira
- Ing. Franco Rupnik

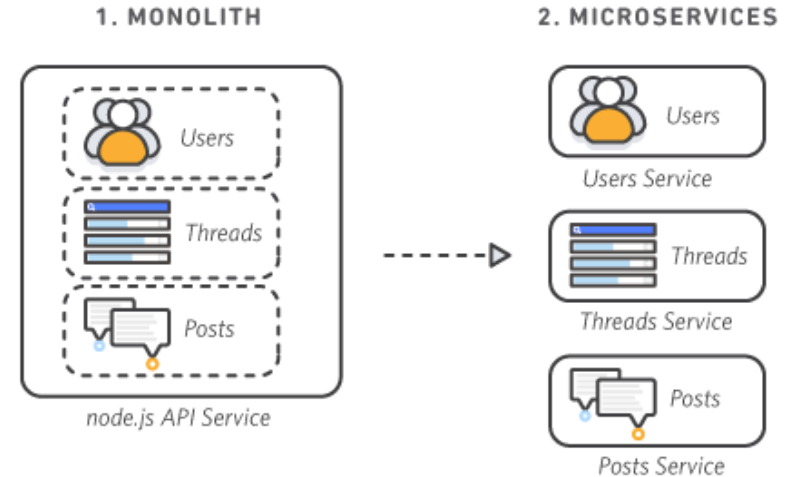
Comunicación Síncrona

Contenidos

1. Síncrono vs. asíncrono: el encuadre
2. REST, gRPC y GraphQL – buenas prácticas y anti-patterns
3. Cómo elegir entre los tres
4. Alternativas emergentes: ConnectRPC
5. Demo: qué pasa cuando la capacidad no alcanza
6. Arquitectura multi-protocolo

¿Cómo hablan los microservicios?

- En un **monolito**, la comunicación se realiza a través de llamadas a funciones internas en memoria.
- En **microservicios**, la comunicación es mediante la red.
- Comunicación por red = latencia, fallos y particiones temporales.
- Elegir el **patrón correcto** de comunicación en microservicios es **clave**.

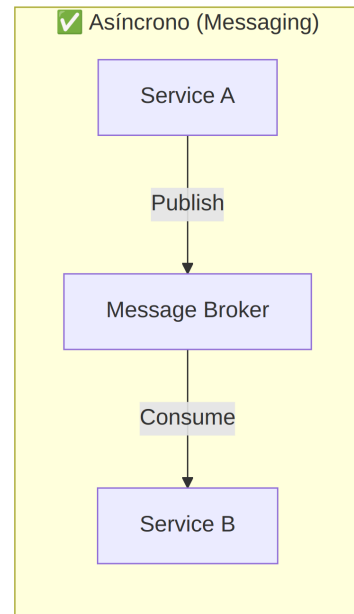
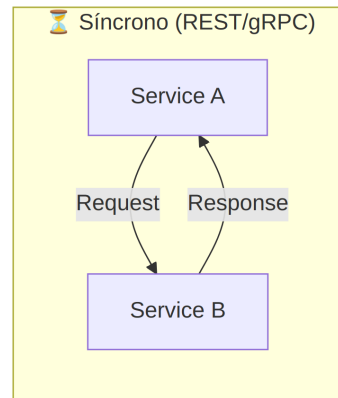


Dos grandes familias

- **Síncrono** = enviás un request y esperás la respuesta.
- **Asíncrono** = “disparás y seguís”; la respuesta (si existe) llega luego.

Ninguno es mejor que el otro, depende.

Hoy vemos **síncrono** en profundidad. Asíncrono y los patrones de coordinación vienen en las próximas dos clases.



Sync vs Async – Comparación

Característica	Síncrono	Asíncrono
Latencia	Baja (respuesta inmediata)	Mayor (broker intermediario)
Código	Simple, flujo lineal	Máquinas de estado, callbacks
Recursos	Proceso bloqueado en espera	Proceso libre, worker duerme
Escalabilidad	Difícil escalar bajo carga	Escala dinámicamente (workers)
Acoplamiento	Temporal: ambos deben estar UP	Espacial: productor no conoce consumidor

Comunicación Síncrona

- **Ciclo:** Request → espera → Response.
- El cliente se **BLOQUEA** hasta recibir respuesta.
- Genera acoplamiento temporal fuerte.
- Si el server se cae, el cliente falla.

Protocolos conocidos: REST (HTTP), gRPC (HTTP/2), GraphQL.

REST: El Estándar Universal

Recursos + verbos HTTP + JSON

Puntos fuertes

- Simple, altamente debuggeable y universalmente soportado.
- Opción excelente y por defecto para desarrollar APIs públicas.
- Tooling maduro: OpenAPI, Postman, cURL, etc.

Puntos débiles

- JSON es verbose = serialización costosa.
- Sin contratos tipados ni streaming nativo.
- OpenAPI es documentación post-hoc más que un contrato.

REST: Buenas Prácticas

- **Versionado desde el día uno:** /api/v1/ en la URL o header Accept-Version.
- **Paginación siempre:** cursor-based > offset (offset se rompe con inserciones concurrentes).
- **Idempotencia en mutaciones:** header Idempotency-Key (patrón Stripe).
- **HTTP status codes correctos:** 201, 404, 409 – no todo es 200 OK con error en el body.
- **HATEOAS selectivo:** links a acciones posibles en el response.
- **Rate limiting + circuit breaker** en APIs públicas.

REST: Anti Patterns Comunes

- **"200 OK" con error en el body:** { "status": 200, "error": "Not Found" }; imposible cachear, imposible filtrar en load balancers.
- **Verbos en la URL:** POST /createOrder en vez de POST /orders; REST son recursos, no funciones.
- **Chatty APIs:** 10 calls para armar una pantalla → considerar BFF o GraphQL.
- **Ignorar ETag / If-None-Match:** sin conditional requests, el cache no sirve.
- **No documentar errores:** el 400 sin body no le dice al consumidor qué corregir.

gRPC: Performance Binaria

Protocol Buffers: binario, tipado, compacto

Puntos fuertes

- HTTP/2: multiplexing, streaming nativo.
- Contrato .proto genera código automáticamente.
- Ideal para service-to-service interno por la velocidad.

Puntos débiles

- No funciona en browsers (requiere proxy).
- HTTP/2 no soportado por load balancers legacy.
- Debugging menos intuitivo que un cURL.
- Obligatorio usar code-gen.

gRPC: Buenas Prácticas

- **Repo único de .protos:** un solo repositorio, versionado con tags; CI genera clientes.
- **Retrocompatibilidad:** nunca borrar ni reenumerar campos; reserved para campos retirados.
- **Deadlines siempre:** timeout explícito en todo call; un servicio lento no traba al caller.
- **Enums con UNKNOWN = 0:** el default de protobuf es 0; sin UNKNOWN no sabés si es intencional.
- **Streaming solo cuando hace falta:** unary es más simple de operar y debuggear.
- **Metadata para tracing:** propagar traceId en headers gRPC, no en el payload.

gRPC: Anti Patterns comunes

- **Olvidar deadlines:** sin timeout, un servicio lento propaga el hang a toda la cadena (cascading failure).
- **Proto files copiados entre repos:** divergencia inevitable; un equipo actualiza, otro no.
- **Payloads enormes en unary:** gRPC tiene límite default de 4 MB; streaming existe por algo.
- **Ignorar reserved:** borrar un campo y reusar el número → deserialización corrupta silenciosa.
- **gRPC para todo:** para APIs públicas o integraciones externas, REST sigue siendo mejor.

GraphQL: Flexibilidad para el cliente

Query language sobre HTTP – pedís exactamente lo que necesitás

Puntos fuertes

- Resuelve over-fetching (REST te da todo) y under-fetching (múltiples calls).
- Schema tipado, introspectable, con tooling maduro.
- Fuerte en BFFs y frontends con datos complejos.

Puntos débiles

- Complejidad (resolvers, N+1 queries).
- Caching más difícil que REST (POST único, no cacheable por URL).
- Seguridad: queries arbitrarias → ataques de profundidad.
- Overhead para APIs simples (CRUD básico = REST gana).

GraphQL: Buenas Prácticas

- **DataLoader para N+1:** sin DataLoader, 10 órdenes con productos = 1 + 10 queries a la DB.
- **Limitar profundidad:** max depth (ej.: 5 niveles) para evitar queries que derriten el server.
- **Complejidad estimada:** asignar "costo" a cada campo, rechazar queries que excedan el límite.
- **Persisted queries en producción:** el frontend envía un hash, no la query completa.
- **Resolvers delgados:** el resolver orquesta, la lógica en services/repositories.
- **Schema-first:** definir el schema SDL como contrato, generar código desde ahí.

GraphQL: Anti-patterns y cuándo NO usarlo

- **GraphQL para CRUD simple:** si tu API es GET /users/:id, REST es más simple y cacheable.
- **GraphQL service-to-service:** para comunicación interna entre microservicios, gRPC gana en performance.
- **Schema gigante sin Federation:** un solo server con 500 types es inmantenible; usar Apollo Federation o schema stitching.
- **Resolver que hace lógica de negocio:** el resolver es un controller, no un service; debe ser delgado.
- **No limitar queries:** sin depth limit + complexity limit, { user { friends { friends { friends... } } } } puede tirar el server.
- **Subscriptions para todo:** websockets son pesados de operar; considerar SSE o polling para datos que cambian poco.

REST vs gRPC vs GraphQL

Característica	REST	gRPC	GraphQL
Formato	JSON / HTTP	Protobuf / HTTP2	JSON / HTTP (queries)
Contrato	OpenAPI (post-hoc)	.proto (genera código)	Schema SDL (tipado)
Fortaleza	Universal, simple	Performance, streaming	Flexibilidad de queries
Ideal para	APIs públicas	Service-to-service	BFF, frontends complejos

No elegís uno para todo, combinás según el caso de uso.

Alternativas Emergentes

¿Qué fricciones tienen REST y gRPC?

Fricción	REST	gRPC
Serialización	JSON verbose, costoso	Protobuf binario, eficiente
Contratos	OpenAPI post-hoc	.proto genera código
Browsers	Funciona nativo	Necesita proxy (Envoy/grpc-web)
Debugging	curl directo	Tooling especial
Streaming	No nativo	HTTP/2 nativo

Cada alternativa ataca una o más de estas fricciones.

ConnectRPC: gRPC sin las partes molestas

Problema: gRPC no funciona en browsers y no se puede testear con curl.

Mismo .proto, un server, **3 protocolos:**

- **Connect** → HTTP/1.1 + JSON (curl, Postman, browsers).
- **gRPC** → HTTP/2 + Protobuf binario (service-to-service).
- **gRPC-Web** → HTTP/1.1 + Protobuf (browsers, sin proxy).

SDK en varios lenguajes: Go, TS, Python, Kotlin, Swift, Dart.



¿Y cuándo no alcanza?

¿Qué pasa cuando llega más gente a lo que a ti le gusta?

Dos letras, y toda la demo sale de ahí

- λ (lambda) – lo que **llega**: requests por segundo.
- μ (mu) – lo que el sistema **puede terminar**: requests por segundo.

Si λ es menor que μ , la espera es transitoria. Si λ supera a μ , el trabajo se acumula y **algo se tiene que romper**: o esperarás, o perdés.

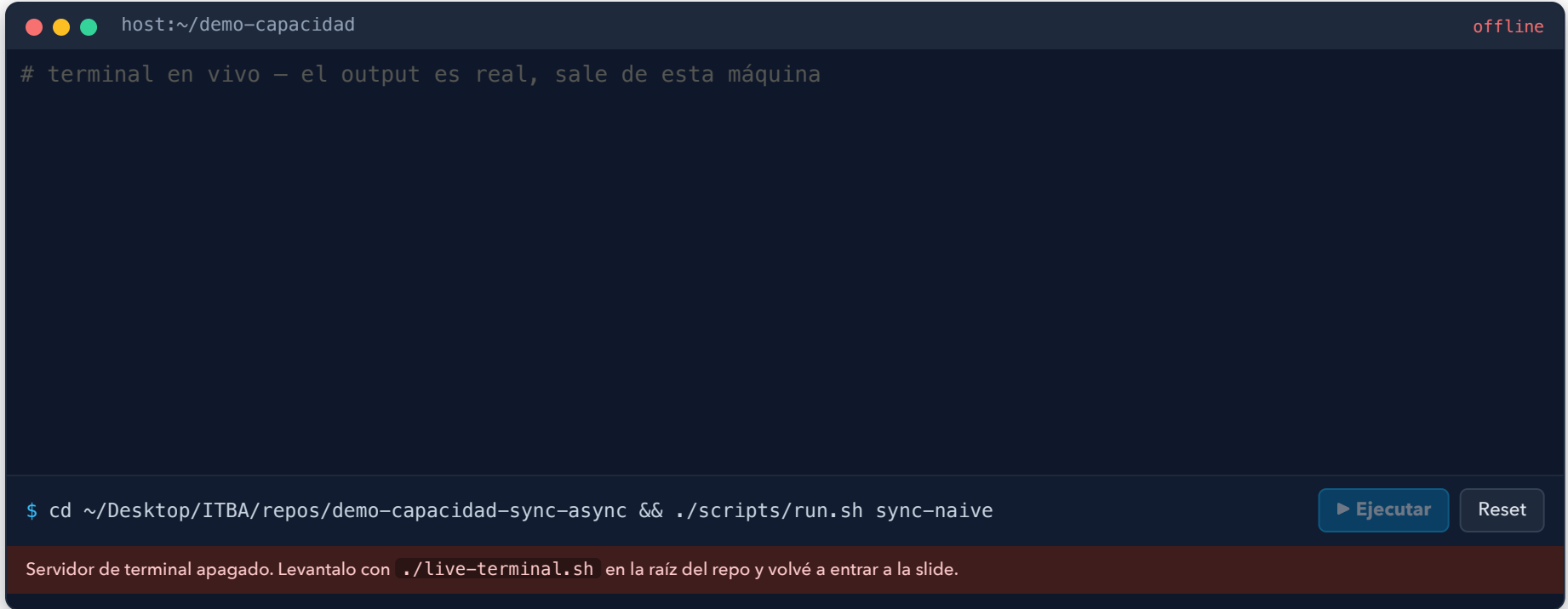
La pregunta de toda la demo es **qué se rompe, y quién paga**.

El experimento

20 clientes	cada uno con 5 cosas que hacer , una tras otra
Closed-loop	no manda la siguiente hasta que vuelve la anterior
2 workers	3 requests en ejecución cada uno
Capacidad	cpus: 0.5 por worker → $\mu \approx 4 \text{ req/s}$
Fallas	20%, y ocurren <i>después</i> de hacer el trabajo
Paciencia	timeout de 3 s, y ahí pasa a la siguiente

Con 3 s de paciencia, 20 clientes demandan **$\approx 6,7 \text{ req/s}$** . El sistema da 4.

Corrida 1: el sync que escribiríamos

A terminal window mockup with a dark blue background. The title bar shows three colored circles (red, yellow, green) and the text 'host:~/demo-capacidad'. The top right corner says 'offline'. The main area contains the text '# terminal en vivo - el output es real, sale de esta máquina'. At the bottom, there is a command prompt '\$ cd ~/Desktop/ITBA/repos/demo-capacidad-sync-async && ./scripts/run.sh sync-naive' followed by two buttons: '▶ Ejecutar' and 'Reset'. A dark red banner at the very bottom contains the text 'Servidor de terminal apagado. Levantalo con ./live-terminal.sh en la raíz del repo y volvé a entrar a la slide.'

```
host:~/demo-capacidad offline
```

terminal en vivo - el output es real, sale de esta máquina

```
$ cd ~/Desktop/ITBA/repos/demo-capacidad-sync-async && ./scripts/run.sh sync-naive
```

Servidor de terminal apagado. Levantalo con `./live-terminal.sh` en la raíz del repo y volvé a entrar a la slide.

Lo que pasó – y lo que siguió pasando

- De **100 pedidos**, completados: **6**.
- Los otros 94: timeout a los 3 s. El cliente se cansó y siguió.
- Memoria del server: de 10 a **93 MiB**.

Y el server siguió trabajando 15 segundos más.

Terminó los 100. Ya no había nadie esperando ninguno.

Lo que le agregamos al sync

Bulkhead – 3 en ejecución más una sala de espera de 2. El sexto se va con 503 al instante, sin tocar la CPU.

Retry – hasta 3 reintentos, con **backoff exponencial y jitter**: 100 ms, 200 ms, 400 ms, cada uno con ruido.

Ningún patrón nuevo y la misma capacidad. Cambia **a quién dejamos entrar y cuánto insiste el que quedó afuera**.

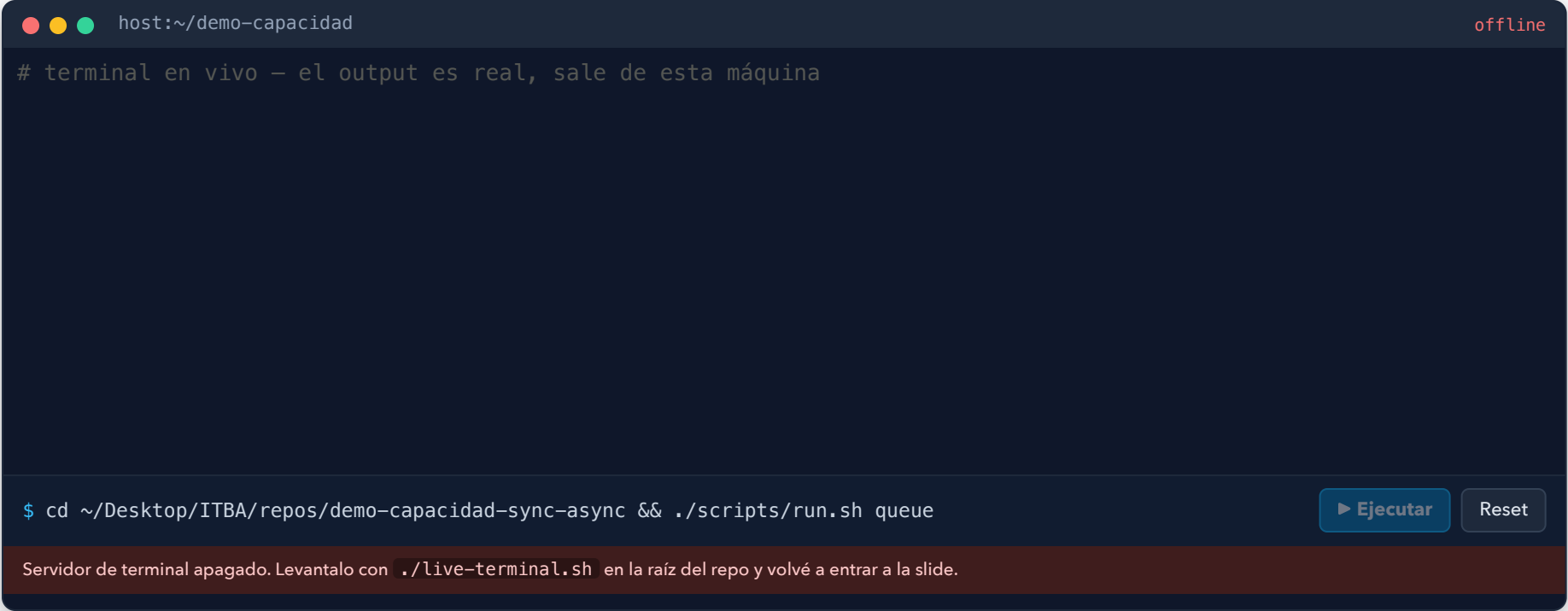
Dos arreglos, del lado sync – y se suman

	Corrida 2: bulkhead	Corrida 3: bulkhead + retry
Qué cambia	sala de espera acotada, 503 inmediato	<i>además</i> , el cliente insiste hasta 3 veces
Completados	21 (de 6)	41
Memoria	93 → 28 MiB	28 MiB
Carga sobre el server	100 intentos	311
Cliente ocupado	9.1 s	20.8 s

Un pedido que sale bien necesitó **1,83 intentos en promedio**. El peor, 4.

El mismo retry **sin** bulkhead: 8 completados y **218 MiB**. Peor que no reintentar.

Corrida 4: una cola en el medio



host:~/demo-capacidad offline

terminal en vivo – el output es real, sale de esta máquina

\$ cd ~/Desktop/ITBA/repos/demo-capacidad-sync-async && ./scripts/run.sh queue

Servidor de terminal apagado. Levantalo con `./live-terminal.sh` en la raíz del repo y volvé a entrar a la slide.

Ejecutar Reset

Las cuatro corridas

	1. naive	2. bulkhead	3. + retry	4. cola
completados	6	21	41	99
– a tiempo (< 3 s)	6	21	41	4
no completados	94	79	59	0
en DLQ	–	–	–	1
intentos / ejecuciones	100	100	311	120
cliente ocupado	15 s	9.1 s	20.8 s	0.05 s
p50 de latencia	3.0 s	1.7 s	1.7 s	17.5 s
pico de memoria	93 MiB	28 MiB	28 MiB	21 MiB
infra extra del broker	–	–	–	0.7 CPU-s · 22 MiB

Misma capacidad, distinto destino del trabajo

Las cuatro corridas son el **mismo binario**, con el **mismo trabajo**, bajo el **mismo medio core**. μ es idéntica **por construcción**.

Lo único que cambia es **qué le pasa a lo que no entra**:

- **Naive** lo pierde, y encima quema CPU en pedidos que ya nadie espera.
- **Bulkhead** lo rechaza rápido y barato: protege al servicio.
- **Retry** lo recupera, si el rechazo es barato: **41 a tiempo**.
- **La cola** no lo pierde nunca, pero lo entrega tarde: **4 a tiempo**.

La cola no agrega capacidad. Convierte pérdida en latencia.

¿Y si le tiro hardware?

La objeción obvia: esto es un rodeo, poné más CPU.

	1 core	4 cores	8 cores	cola, 1 core
completados	5	88	79	99
timeouts	94	0	0	0
trabajo huérfano	17 s	3 s	4 s	–
pico de memoria	89 MiB	53 MiB	53 MiB	21 MiB

Funciona, y conviene decirlo así de claro en vez de venderles la cola.

Pero pagás ese hardware las 24 horas para un pico que dura segundos. Y la memoria baja a medias –89 a 53 MiB– contra los **28 del bulkhead**, que no costó nada.

¿Y cuando el pedido falla?

El 20% falla **después** de hacer el trabajo. Alguien tiene que decidir qué hacer con él.

- **Reintentar** – con backoff exponencial y jitter
- **Idempotencia** – el trabajo ya se hizo una vez
- **DLQ** – tras N intentos, apartar en vez de perder
- **Circuit breaker** – si falla siempre, dejar de intentar

Sync lo decide el cliente, con el reloj. La cola lo decide el broker, con intentos.

Lo que esta demo NO prueba

- La cola era **en memoria**: si el broker se muere, se pierde todo el backlog.
- Te salvó de la **sobrecarga**, no de un **crash**.
- El broker puede entregar el mismo mensaje **dos veces**.
- Con $\lambda > \mu$ **sostenida**, ninguna cola alcanza: el backlog llega al techo y el rechazo vuelve.

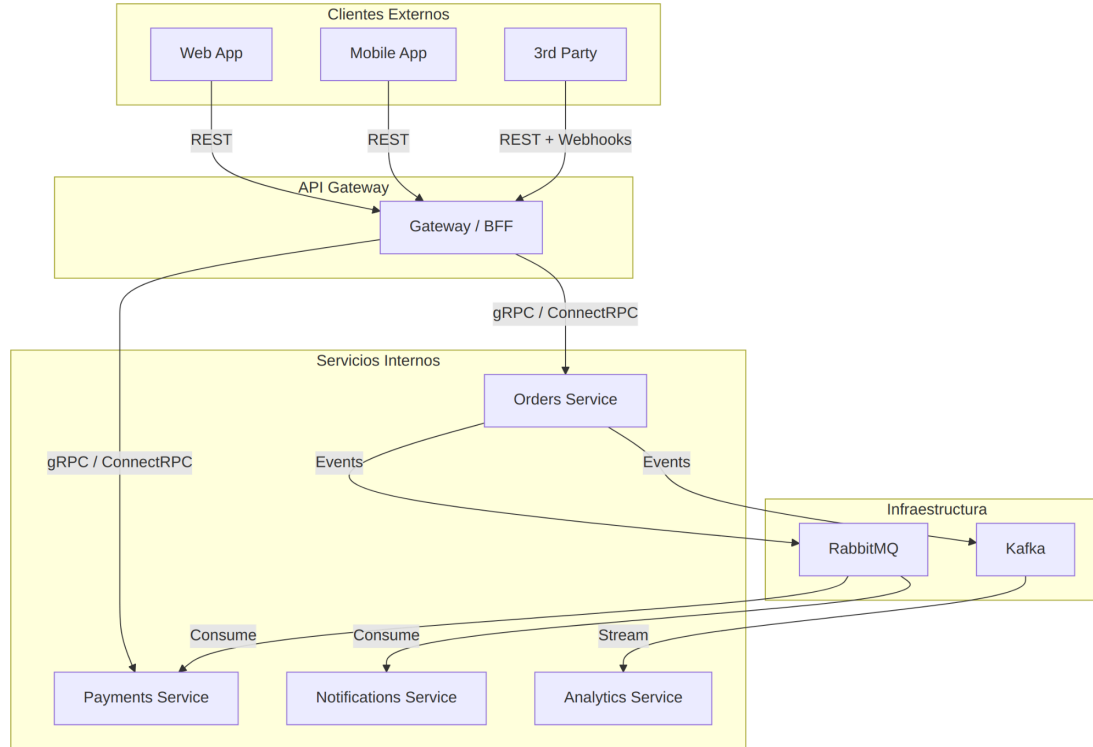
Persistencia, garantías de entrega e idempotencia: **clase 7**.

Multi-protocolo: La arquitectura ganadora

- **REST** → APIs públicas, externas.
- **gRPC/ConnectRPC** → interno, baja latencia.
- **Message Queue** → eventos, procesamiento async.
- **GraphQL** → BFF, frontends con datos complejos.

No existe un protocolo maestro. La arquitectura óptima combina opciones según el problema a resolver.

Multi-protocolo: Ejemplo



Recap: ¿Qué nos llevamos?

- Síncrono vs asíncrono: no hay mejor, hay trade-offs.
- REST público, gRPC interno, GraphQL para BFF.
- El contrato es lo que importa: OpenAPI, .proto, schema SDL.
- Deadlines, versionado e idempotencia no son opcionales.
- ConnectRPC ataca una fricción concreta, no reemplaza a gRPC.
- Bajo sobrecarga: sync pierde trabajo, la cola acumula latencia.
- Multi-protocolo: combinás según el caso de uso.

Lo que viene: **asíncrono y EDA** (clase 7) y **estado distribuido y Saga** (clase 8).

Fin!!